



Módulo 7

Transformación y modelado con Python/Pandas

Curso: Data Engineer Básico

3 horas

Objetivos de Aprendizaje

- **Aplicar** técnicas de limpieza de datos (valores nulos, duplicados, tipos) en datasets reales usando Pandas
- **Ejecutar** operaciones de join (inner, left, right, full outer) para combinar múltiples fuentes de datos
- **Calcular** KPIs de disponibilidad y rendimiento utilizando agregaciones y funciones de ventana
- **Generar** datasets en capas Silver y Gold siguiendo las buenas prácticas de arquitectura medallón

1. Limpieza de Datos con Pandas

30 min

Inspección inicial de datasets

Antes de cualquier transformación, es esencial comprender la **calidad de los datos**:

```
df.info()      # Tipos de datos, memoria, valores nulos
df.describe()  # Estadísticas descriptivas (numéricas)
df.head(10)    # Vista preliminar
df.shape       # Dimensiones: (filas, columnas)
```

Tratamiento de valores nulos

Estrategias según el contexto:

Estrategia	Método Pandas	Cuándo usarlo
Eliminar filas	<code>df.dropna()</code>	Pocos nulos, datos no críticos
Imputar media	<code>df.fillna(df['col'].mean())</code>	Datos numéricos simétricos
Imputar mediana	<code>df.fillna(df['col'].median())</code>	Datos con outliers
Forward fill	<code>df.fillna(method='ffill')</code>	Series temporales

Manejo de duplicados

```
# Detectar duplicados
df.duplicated().sum()
df.duplicated(subset=['col1', 'col2']).sum()

# Eliminar duplicados
df.drop_duplicates(inplace=True)
df.drop_duplicates(subset=['col1', 'col2'], keep='first')
```

⚠ **Error común:** No verificar duplicados antes de agregaciones puede generar totales incorrectos.

Conversión de tipos y normalización

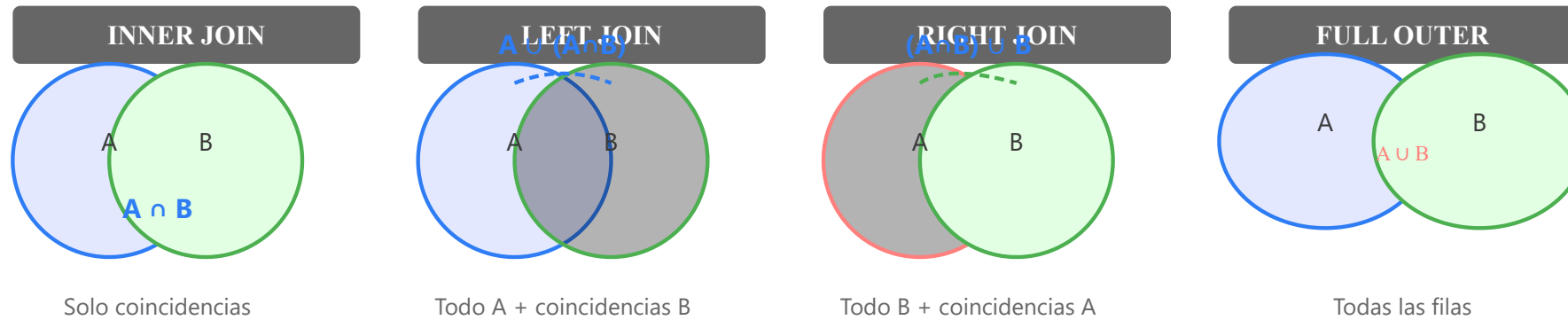
```
# Convertir tipos
df['fecha'] = pd.to_datetime(df['fecha'])
df['importe'] = pd.to_numeric(df['importe'], errors='coerce')

# Normalizar texto
df['nombre'] = df['nombre'].str.strip().str.lower()
df['nombre'] = df['nombre'].str.replace(r'^\w\s', '', regex=True)
```

2. Joins entre DataFrames

30 min

Tipos de Join



pandas.merge()

```
resultado = pd.merge(df_izq, df_der, on='columna', how='inner')
```

Sintaxis: on=clave, how=tipo_join, suffixes=('_izq','_der')

Parámetros clave: left_on, right_on para columnas con nombres diferentes

Sintaxis pandas.merge()

pandas.merge() - Sintaxis

```
resultado = pd.merge(  
    df_izquierda,      # DataFrame izquierdo  
    df_derecha,        # DataFrame derecho  
    on = 'columna_clave' # Columna para JOIN  
    how = 'inner'      # inner, left, right, outer  
    suffixes = ( '_izq' , '_der' ) # Sufijos para columnas duplicadas  
)  
  
# left_on / right_on para columnas con nombres diferentes
```

Ejemplo práctico: Orders + Inventory

```
ordenes = pd.read_csv('ordenes_mantenimiento.csv')
inventario = pd.read_csv('inventario_activos.csv')

ordenes_enriquecidas = pd.merge(
    ordenes,
    inventario,
    left_on='activo_id',
    right_on='id_activo',
    how='left',
    suffixes=('', '_activo')
)
```

Tip: Usa `left_on` y `right_on` cuando las columnas de join tienen nombres diferentes.

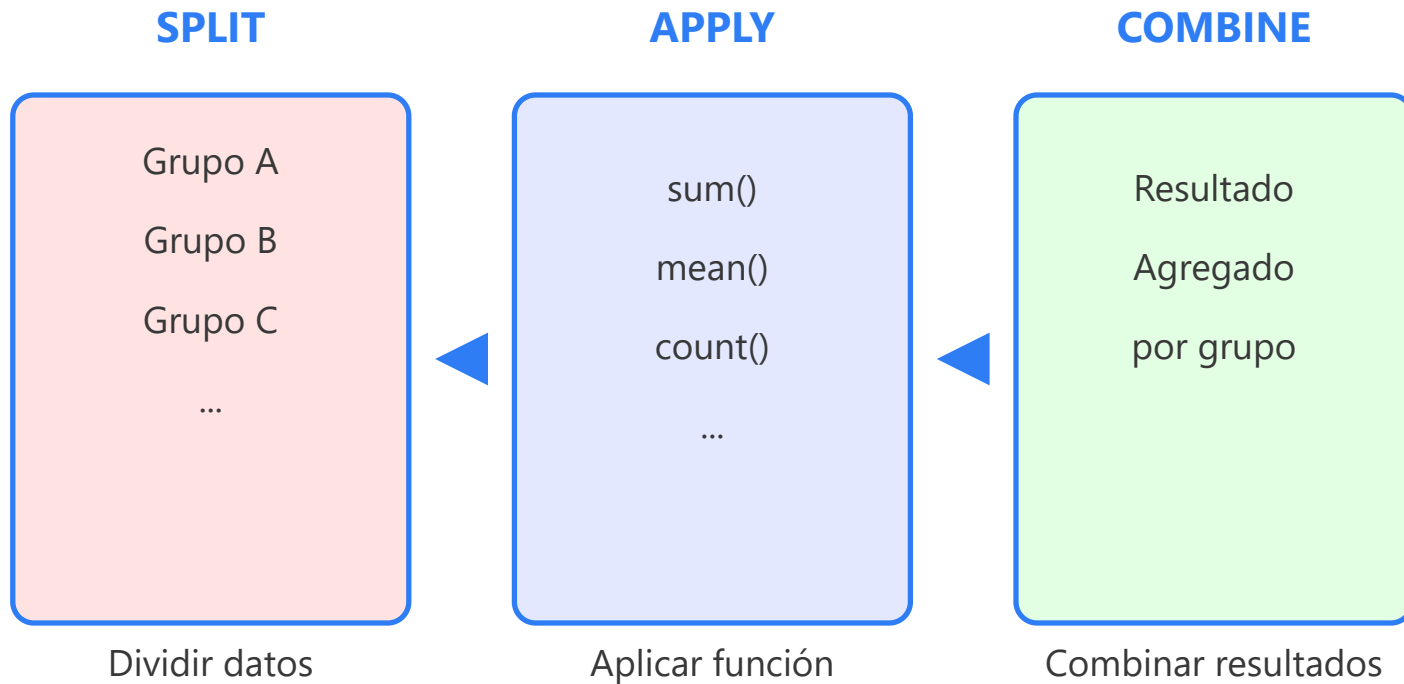
Concatenación vs Merge

Operación	Cuándo usarla
<code>pd.concat([df1, df2])</code>	Apilar filas del mismo esquema
<code>pd.merge(df1, df2)</code>	Combinar por clave, esquemas diferentes

3. Agregaciones y GroupBy

25 min

Concepto: Split - Apply - Combine



Estructura del GroupBy

```
df.groupby('columna')['valor'].función()  
  
# Ejemplo: Total de mantenimiento por tipo de activo  
df.groupby('tipo_activo')['coste'].sum()
```

Funciones de agregación comunes

Función	Descripción
<code>sum()</code>	Suma de valores
<code>mean()</code>	Promedio
<code>median()</code>	Mediana
<code>count()</code>	Conteo de valores no nulos
<code>nunique()</code>	Conteo de valores únicos
<code>std()</code> / <code>var()</code>	Desviación estándar / Varianza

Múltiples agregaciones

```
resumen = df.groupby('tipo_activo').agg({
    'coste': ['sum', 'mean', 'max'],
    'orden_id': 'count'
})
resumen.columns = ['coste_total', 'coste_promedio', 'coste_max', 'num_ordenes']
```

4. Funciones de Ventana

25 min

Concepto de Window Functions

Las funciones de ventana realizan cálculos sobre un conjunto de filas relacionadas con la fila actual, **manteniendo la granularidad original**.

```
df['columna_ventana'] = df['columna'].rolling(window=3).mean()
```

Tipos de ventanas

Tipo	Descripción	Ejemplo
Rolling	Ventana deslizante fija	Últimas 3 filas
Expanding	Acumulativa desde inicio	Desde primer registro
Exponential Weighted	Ponderación exponencial	Media móvil con decay

Ejemplos con rolling window

```
# Media móvil de 7 días
df['coste_ma_7'] = df['coste'].rolling(window=7).mean()

# Total acumulado
df['coste_cumsum'] = df['coste'].cumsum()

# Máximo de los últimos 5 registros
df['coste_max_5'] = df['coste'].rolling(window=5).max()
```

Lag y Lead

```
# Valor anterior (lag)
df['coste_anterior'] = df.sort_values('fecha').groupby('activo_id')['coste'].shift(1)

# Valor siguiente (lead)
df['coste_siguiente'] = df.sort_values('fecha').groupby('activo_id')['coste'].shift(-1)
```

Uso: Comparar valores consecutivos, calcular tendencias, detectar cambios.

5. Introducción a Spark DataFrames

20 min

¿Qué es Apache Spark?

Framework de procesamiento distribuido para datos grandes:

- **Velocidad:** Procesamiento en memoria
- **Escalabilidad:** De laptops a clusters de miles de nodos
- **Versatilidad:** Batch, streaming, ML, SQL

Pandas vs Spark

Aspecto	Pandas	Spark
Volumen datos	Un solo nodo (GB)	Distribuido (TB/PB)
Latencia	Baja	Mayor
API	<code>DataFrame</code>	<code>DataFrame</code> / <code>Dataset</code>
Ejecución	Inmediata	Lazy (transformaciones)

Spark DataFrame básico

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("TransCore").getOrCreate()

df = spark.read.csv('ordenes_mantenimiento.csv', header=True, inferSchema=True)

df.show(5)
df.printSchema()
df.select('activo_id', 'coste').filter(df['estado'] == 'completada').show()
```

Cuándo usar Pandas vs Spark

Usar Pandas cuando:

- Dataset cabe en memoria RAM (< 10 GB)
- Prototipado rápido
- Análisis exploratorio

Usar Spark cuando:

- Datos distribuidos en cluster
- Pipeline de producción
- Necesidad de paralelización real

Lab: Cálculo de KPIs y Generación Silver/Gold

45 min - Case Study TransCore

Objetivo de la Práctica

Utilizando los datasets `ordenes_mantenimiento.csv` e `inventario_activos.csv`:

1. Limpiar y preparar los datos
2. Unir ambas fuentes mediante joins apropiados
3. Calcular KPIs de disponibilidad y rendimiento
4. Generar datasets en capa Silver
5. Producir vistas agregadas en capa Gold

Datasets de Entrada

ordenes_mantenimiento.csv

```
orden_id,activo_id,fecha_creacion,fecha_ejecucion,estado,tipo_intervencion,coste,tecnico
ORD001,ACT001,2025-01-15,2025-01-20,completada,preventivo,450.00,Javier
ORD002,ACT002,2025-01-18,2025-01-22,completada,correctivo,780.00,Ana
```

inventario_activos.csv

```
id_activo,nombre_activo,tipo_activo,ubicacion,fecha_adquisicion,valor
ACT001,Compresor Principal,Compresor,Nave A,2018-05-10,15000
ACT002,Grúa Torre 1,Grúa,Nave B,2019-03-20,12000
```

KPI 1: Disponibilidad por Activo

Fórmula:

$$\text{Disponibilidad} = \frac{\text{Días operativos}}{\text{Días totales del período}} \times 100$$

```
def calcular_disponibilidad(activo_id, ordenes_df, inventario_df):
    activo_info = inventario_df[inventario_df['id_activo'] == activo_id].iloc[0]
    fecha_adq = pd.to_datetime(activo_info['fecha_adquisicion'])
    ultima_orden = ordenes_df[(ordenes_df['activo_id'] == activo_id) &
                              (ordenes_df['estado'] == 'completada')]['fecha_ejecucion'].max()
    dias_totales = (ultima_orden - fecha_adq).days
    dias_inactivo = len(ordenes_df[(ordenes_df['activo_id'] == activo_id)])
    return ((dias_totales - dias_inactivo) / dias_totales) * 100 if dias_totales > 0 else 0
```

KPI 2 y 3: Coste y Cumplimiento

```
# KPI 2: Coste total de mantenimiento por activo
coste_por_activo = ordenes.groupby('activo_id')['coste'].sum()

# KPI 3: Coste por tipo de intervención
coste_por_tipo = ordenes.groupby('tipo_intervencion')['coste'].sum()

# KPI 4: Tasa de cumplimiento
total_ordenes = len(ordenes)
ordenes_completadas = len(ordenes[ordenes['estado'] == 'completada'])
tasa_cumplimiento = (ordenes_completadas / total_ordenes) * 100
```


Generación de Capa Silver

Silver y Gold: Código

```
# SILVER: Datos enriquecidos y limpios
ordenes_silver = ordenes_enriquecidas.copy()
ordenes_silver['disponibilidad_kpi'] = ordenes_silver['activo_id'].map(disponibilidad_kpi)
ordenes_silver.to_parquet('silver/ordenes_mantenimiento_silver.parquet', index=False)

# GOLD: Vistas agregadas optimizadas para negocio
kpis_activo = ordenes.groupby('activo_id').agg({
    'coste': 'sum', 'orden_id': 'count'
}).rename(columns={'orden_id': 'total_ordenes'})
kpis_activo['disponibilidad'] = kpis_activo.index.map(disponibilidad_kpi)
kpis_activo.to_parquet('gold/kpis_por_activo_gold.parquet', index=False)
```

Errores comunes y soluciones

Error	Causa	Solución
<code>KeyError</code> en merge	Nombres de columna diferentes	Verificar <code>left_on</code> y <code>right_on</code>
Valores nulos en KPIs	Fechas no convertidas	Usar <code>pd.to_datetime()</code> antes de cálculos
join produce filas inesperadas	Tipo de dato diferente	Asegurar mismo dtype en claves
<code>ZeroDivisionError</code>	División por cero	Validar denominador antes de dividir

Resumen

Puntos Clave

- **Limpieza de datos:** El 80% del trabajo de un data engineer es limpiar y preparar datos. Pandas ofrece herramientas poderosas para esta tarea.
- **Joins:** La elección del tipo de join (inner, left, right, outer) afecta significativamente los resultados. Siempre verificar después de unir.
- **Agregaciones:** `groupby` es fundamental para resumir información. Dominar las múltiples agregaciones es esencial.
- **Funciones de ventana:** Permiten cálculos sobre secuencias sin perder la granularidad original.
- **Arquitectura Medallón:** Bronze = datos crudos, Silver = datos confiables, Gold = datos listos para negocio.

Fórmulas importantes

Disponibilidad: $\frac{\text{Días operativos}}{\text{Días totales}} \times 100\%$

Tasa de cumplimiento: $\frac{\text{Completadas}}{\text{Total}} \times 100\%$

Referencias

Recursos Adicionales

- [Pandas Documentation](#)
- [PySpark Documentation](#)

Lecturas Recomendadas

- *Data Engineering with Python* - Paul Crickard
- *The Data Warehouse Toolkit* - Ralph Kimball

Herramientas

Herramienta	Uso
Jupyter Notebook	Prototipado y exploración
VS Code + Python	Desarrollo de scripts
Apache Spark	Procesamiento distribuido
DBeaver	Exploración de datos



Gracias!

Módulo 7 Completado

Siguiente: Módulo 8 - Catalogación y gobierno de datos